Reporte Técnico de Actividades Práctico-Experimentales Nro. 2

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Richard Cajas Ismael Gonzalez Jostin Vasquez Miguel Veintimilla Gyna Yupanqui
Asignatura	Sistemas Digitales
Ciclo	5to
Unidad	1 – Microcontroladores para sistemas embebidos
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Juego de luces interactivo con ESP32
Nombre del Docente	Luis Darío Sinche C
Fecha	23 de octubre de 2026
Horario	Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (simulación en Wokwi y prueba en la placa ESP32 del grupo)
Tiempo planificado en el Sílabo	3 horas (APE, semana 2): 2 h presenciales y 1 h para terminar el informe

2. Objetivo(s) de la Práctica

Programar en el ESP32 un juego de luces con cuatro LEDs cuya velocidad cambia con un pulsador, usando temporización no bloqueante con millis() y la lectura del pulsador con pull-up interna, antirrebote y detección de flanco.

3. Materiales, Reactivos, Equipos y Herramientas

Cantidad	Componente
1	ESP32 DevKit V1 de 30 pines + cable USB de datos (no solo de carga)
1	Protoboard

4	LEDs de 5 mm rojos, amarillos o verdes (no azules ni blancos: casi no encienden a 3,3 V)
4	Resistencias de 330 Ω
1	1 Pulsador de 4 patas
6	Cables jumper macho-macho

Software (cada estudiante):

- Wokwi (wokwi.com), en el navegador, sin instalar nada.
- Arduino IDE 2.x con el paquete esp32 by Espressif Systems versión 3.x, para la Parte C.
- Cuenta de GitHub y acceso al EVA, para la Parte D.

4. Procedimiento / Metodología Ejecutada

La práctica se desarrolló en dos fases combinando simulación virtual y pruebas en hardware real.

4.1. Inicialmente, se implementó el circuito en la plataforma Wokwi conectando cuatro LEDs y un pulsador a los pines GPIO de una placa ESP32. Mediante el IDE de Arduino (usando el paquete oficial de esp32),

4.2. Se programó una secuencia de luces utilizando la función `millis()` en lugar de `delay()` para garantizar una temporización no bloqueante.

4.3. Se integró el control del pulsador utilizando la resistencia pull-up interna del microcontrolador (`INPUT_PULLUP`) y un algoritmo de antirrebote por software para ajustar dinámicamente la velocidad de los LEDs.

4.4. Finalmente, el código validado se cargó en un microcontrolador ESP32 físico, armado sobre una protoboard, para comprobar su fiabilidad y respuesta mecánica en un entorno real.

5. Resultados

Incluya evidencias del trabajo realizado (tablas, gráficos, capturas de pantalla, registros de ejecución, modelos, programas, informes, etc.).

Figura 1: A4 periodo:300

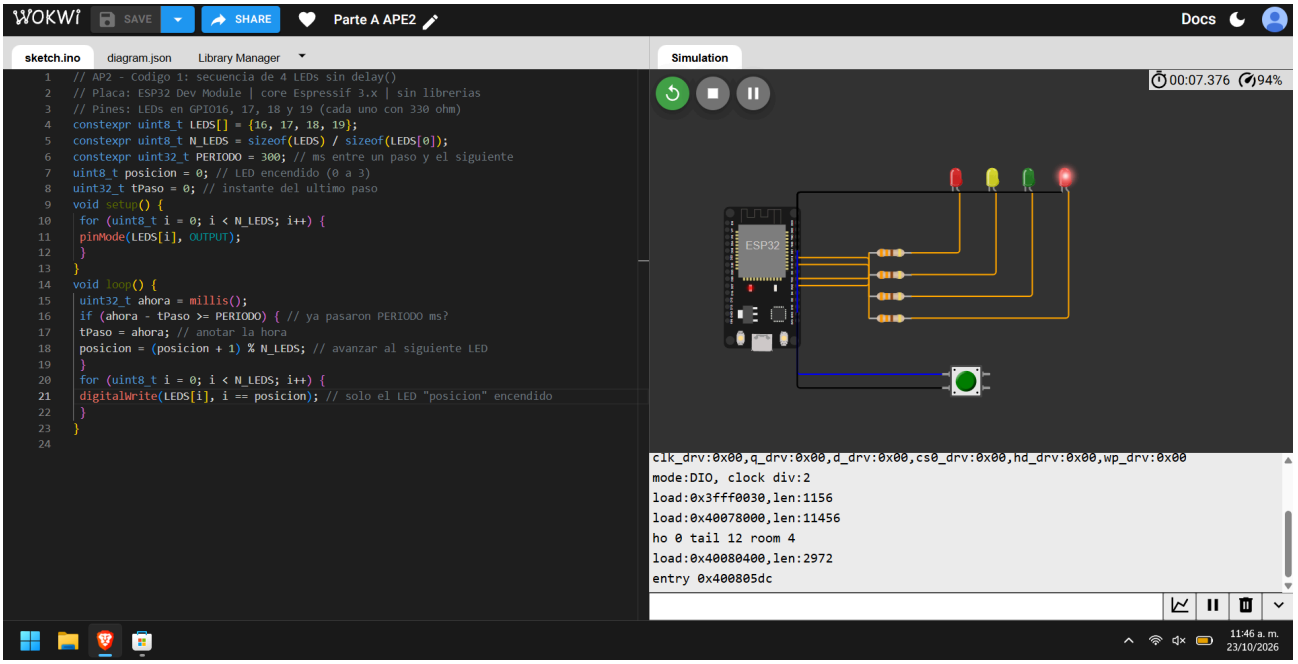


Figura 2: A5 periodo: 1000

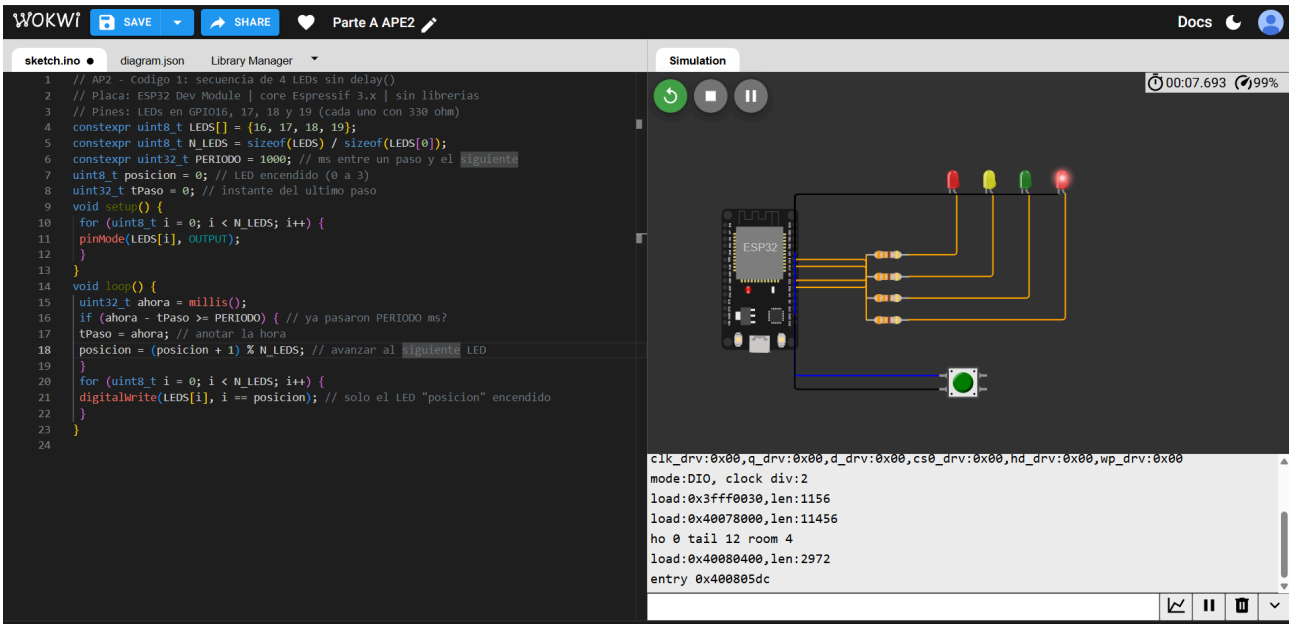


Figura 3: A5 periodo:100

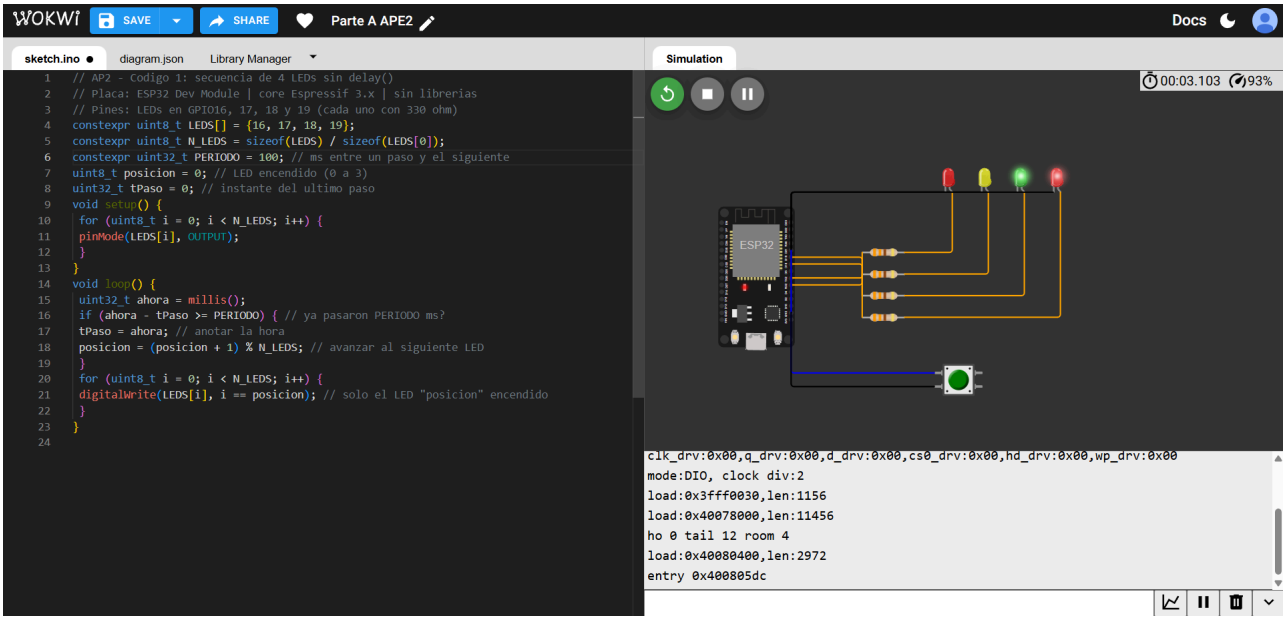


Tabla A. Efecto del periodo

PERIODO (ms)	¿Qué observan?
1000	La luz pasa de un LED a otro bastante despacio. Cada LED se queda encendido un segundo completo antes de cambiar al siguiente.
300	El cambio entre los LEDs se nota fluido y a buena velocidad. No va ni muy rápido ni muy lento, se distingue fácil el recorrido.
100	Van muy rápido. Las luces cambian tan seguido (diez veces por segundo) que casi parece un parpadeo o una ráfaga continua.

Figura 4: B1,B2 y B3

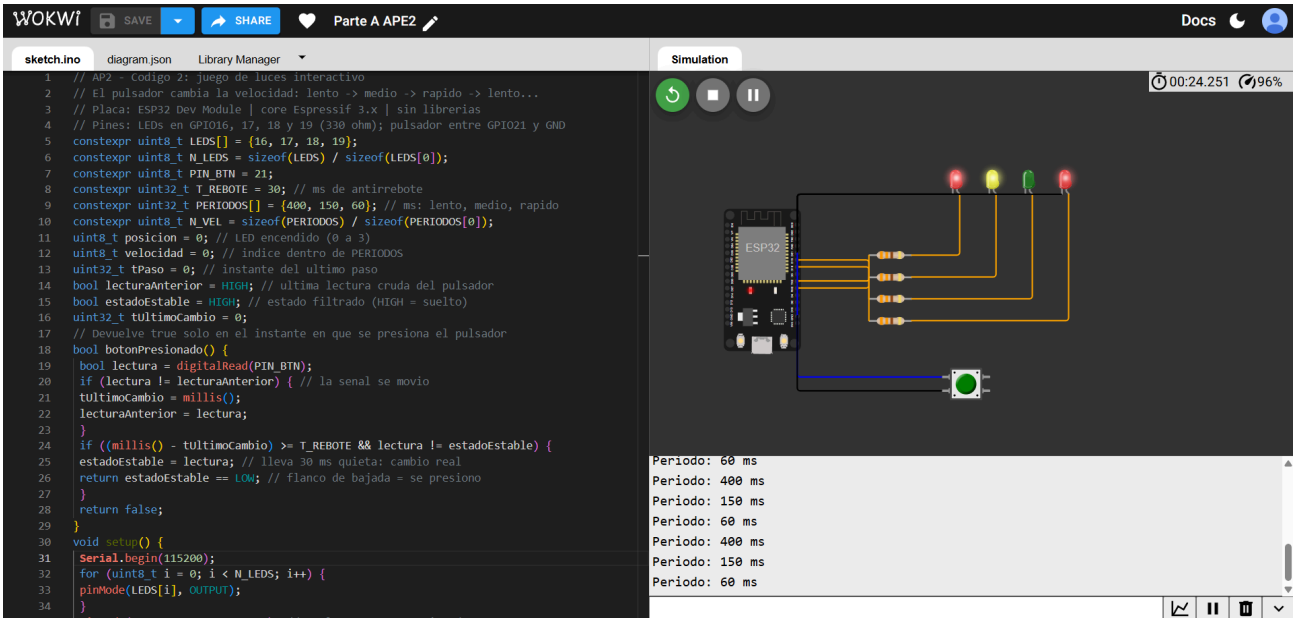


Figura 5: B4

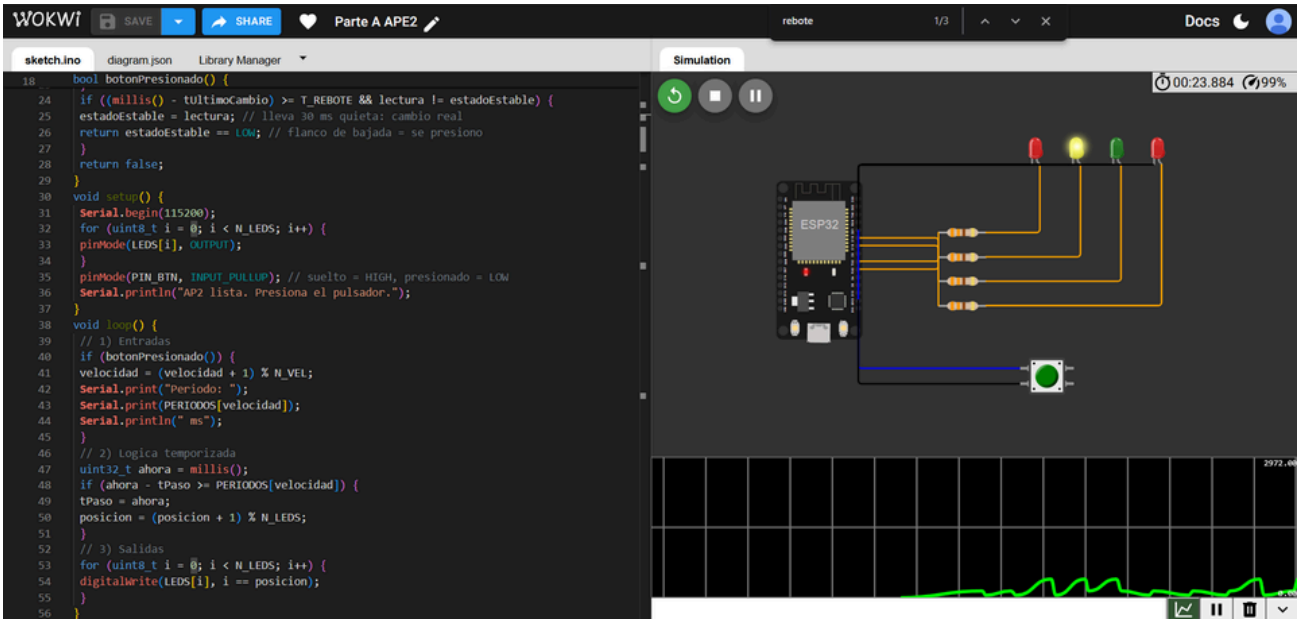


Figura 6: B5 y B6

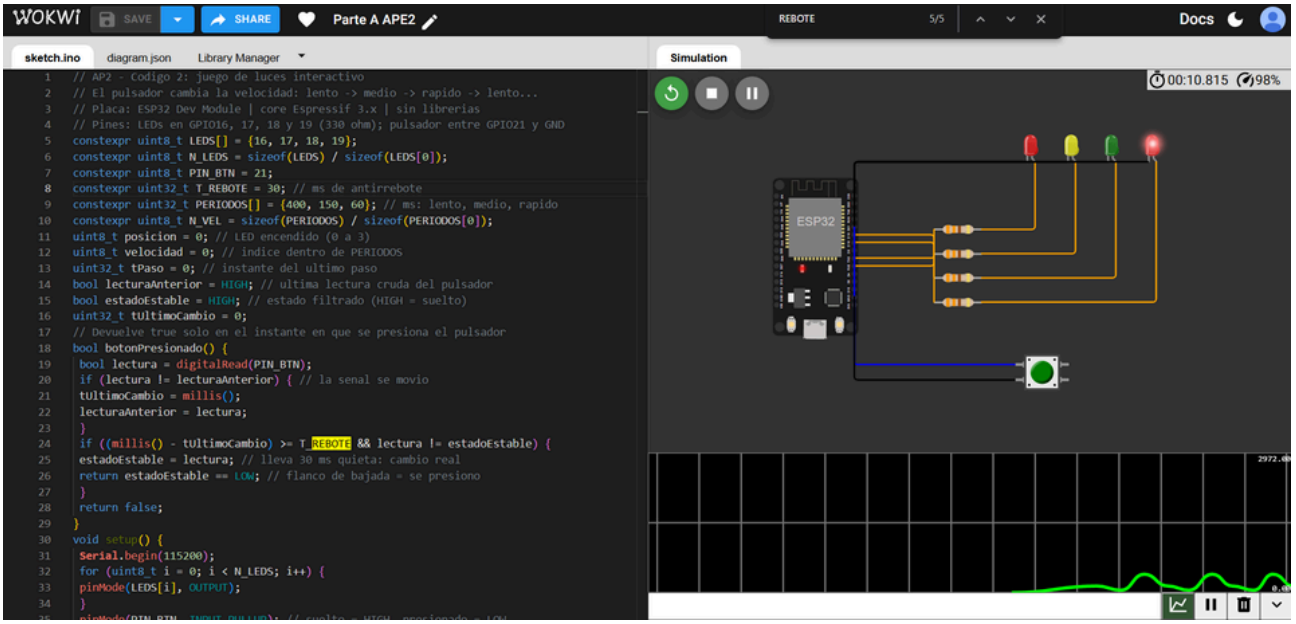
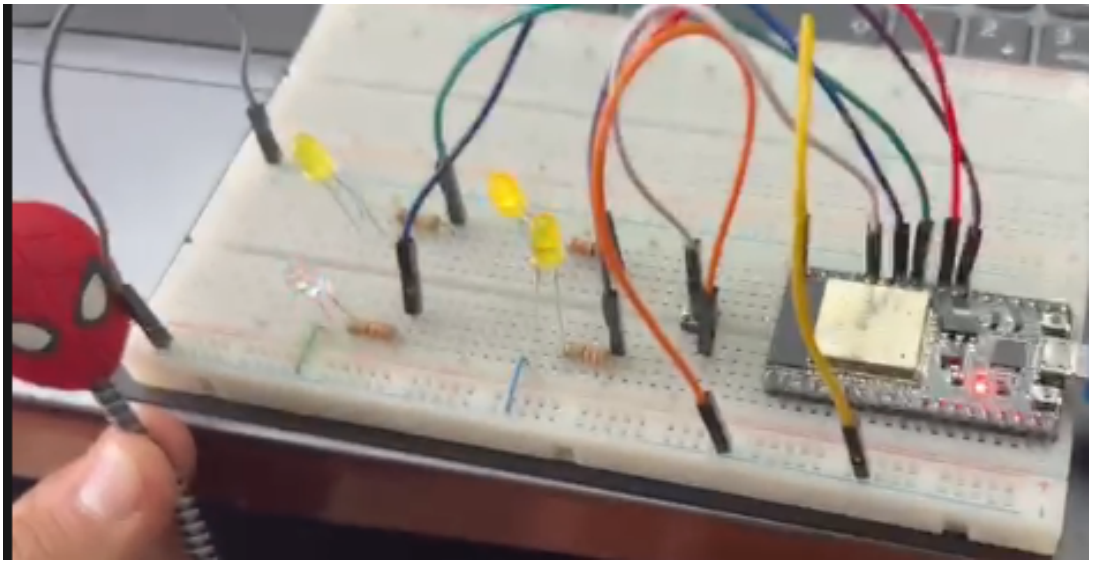


Tabla B. Efecto del antirrebote (10 clics)

T_REBOTE (ms)	Líneas impresas	¿Coincide con los 10 clics?
0	Más de 10	**No.** Como se quitó el tiempo de espera, al presionar el botón una sola vez la tarjeta detecta varios toques falsos por la vibración del interruptor.
30	10	**Sí.** Con este tiempo de pausa el sistema ignora esos pequeños saltos de señal y registra exactamente una línea por cada toque que le damos.

Figura 7: Evidencia del circuito armado en físico.



6. Preguntas de Control

6.1. ¿Por qué el Código 1 usa millis() en lugar de delay()? ¿Qué pasaría con el pulsador del Código 2 si cada paso de los LEDs usara delay(400)?

Se utiliza millis() porque proporciona una temporización no bloqueante, lo que permite que el microcontrolador siga ejecutando el ciclo loop() y atendiendo otras tareas. Si se usara delay(400), el procesador se pausaría completamente durante esos 400 ms; cualquier pulsación que ocurriera durante ese lapso sería ignorada, resultando en un sistema ineficiente que no respondería de inmediato.

6.2. El pulsador va entre el GPIO21 y GND, sin resistencia externa. ¿Qué valor lee digitalRead() con el pulsador suelto y con el pulsador presionado? ¿Por qué?

Estando suelto, lee HIGH (o 1) porque el pin está configurado como INPUT_PULLUP, lo que lo conecta internamente a los 3.3V a través de una resistencia. Estando presionado, lee LOW (o 0) porque el circuito se cierra llevando directamente el pin a GND (0V).

6.3. ¿Qué pasaría si conectaran el pulsador al GPIO34 sin cambiar el código? Justifiquen con lo visto en clase.

El GPIO34 es un pin exclusivo de entrada y no posee resistencia pull-up interna. La instrucción INPUT_PULLUP no tendría efecto, dejando el pin en un estado "flotante" cuando el botón está suelto. Esto provoca lecturas aleatorias debido al ruido electromagnético, causando que la velocidad del juego de luces cambie de forma incontrolable.

6.4. En el paso B3 la velocidad cambió una sola vez aunque mantuvieron presionado el pulsador. ¿Qué parte del Código 2 lo garantiza?

Lo garantiza la lógica del bloque if ((millis() - tUltimoCambio) >= T_REBOTE && lectura != estadoEstable) actualiza el estado estable, y la función solo retorna true justo cuando se detecta el flanco de bajada (de HIGH a LOW). Esto hace que el cambio de velocidad se dispare una sola vez por cada vez que se aplasta el botón, ignorándolo mientras se mantiene pulsado.

6.5. Expliquen la Tabla B: ¿por qué con T_REBOTE = 0 puede haber más líneas impresas que clics?

Se debe al "ruido mecánico". Los contactos metálicos internos del pulsador rebotan o vibran en escala microscópica antes de asentarse. Con un T_REBOTE = 0, el microcontrolador lee a gran velocidad todos esos micro-contactos como si fueran varias pulsaciones físicas distintas por cada vez que el usuario presiona el botón.

6.6. Calculen la corriente que circula por un LED rojo (Vf ≈ 2,0 V) con una resistencia de 330 Ω a 3,3 V. ¿Por qué no conviene usar un LED azul?

Por Ley de Ohm:

$$I = (V_{fuente} - V_{led})/R.$$
$$I = (3,3\sim V - 2,0\sim V)/330\sim\Omega = 1,3\sim V/330\sim\Omega \approx 0,00394\sim A = 3,94\sim mA.$$

No conviene usar un LED azul porque su voltaje de polarización directa (Vf) suele estar entre 3,0 V y 3,3 V. Al alimentarlo con 3,3 V, apenas quedaría voltaje remanente en la resistencia, provocando que el LED brille muy poco o simplemente no pueda prenderse.

6.7. Mencionen una diferencia entre probar el circuito en Wokwi y probarlo en la placa real.

En Wokwi, las conexiones son ideales, los cables no sufren falsos contactos y los botones siempre se comportan uniformemente (incluso el rebote es simulado numéricamente). En la placa real, intervienen imperfecciones físicas, cables sueltos en la protoboard, ruidos electromagnéticos reales y puede requerirse la pulsación del botón BOOT físico en la placa para que Arduino IDE suba el programa al ESP32.

6.8. El ESP32 sigue funcionando a 240 MHz aunque el juego solo cambie de LED cada 400 ms. Propongan una idea para reducir el consumo de energía si el juego funcionara con batería.

Una solución sería usar funciones de bajo consumo del microcontrolador, como bajar la frecuencia del reloj del procesador (por ejemplo, a 80 MHz en lugar de 240 MHz), o aprovechar la temporización para colocar el ESP32 en modo *"Light Sleep"* en el tiempo de espera entre parpadeos, despertándolo mediante una interrupción de timer o la interrupción por hardware del pin del pulsador.

7. Conclusiones

- La sustitución de la instrucción delay() por el control de tiempo mediante millis() garantiza un comportamiento "no bloqueante". Esto permitió que el programa gestionara de manera simultánea e inmediata tanto la secuencia periódica de encendido de LEDs como la atención en tiempo real al botón del usuario.
- El uso del parámetro INPUT_PULLUP evidencia la ventaja de utilizar las características integradas de hardware del microcontrolador ESP32, lo que simplifica el ensamblaje en protoboard y minimizando la cantidad de componentes necesarios como resistencias externas.
- La implementación del algoritmo de antirrebote (debouncing) por software debe solucionar problemas derivados de los límites físicos del hardware (el rebote metálico de los botones), lo que asegura una captura limpia de la pulsación para una navegación fluida entre velocidades.

8. Recomendaciones

Al pasar del simulador a la placa real, se recomienda siempre cruzar la información del esquema diagram.json con la serigrafía real de la placa ESP32 del grupo, debido a que diferentes versiones clónicas varían el orden de pines (por ejemplo, la ubicación de TX2 y RX2).

En prácticas futuras donde los requerimientos de interacción aumenten o requieran aún mayor precisión en la entrada del usuario, se sugiere investigar y emplear interrupciones por hardware (Interrupts) en lugar del método de sondeo constante (polling) usado en el ciclo loop.

Al armar los circuitos, evitar forzar los cables DuPont y las patas de los LEDs dentro de la protoboard; un falso contacto puede derivar en lecturas aleatorias en los pines y generar comportamientos inesperados de temporización que sean difíciles de rastrear solo analizando el código de programación.

9. Reflexión crítica

El desarrollo de esta práctica permitió generar un entendimiento profundo sobre el concepto de concurrencia y multitarea en hardware mononúcleo. Tradicionalmente, suele ser fácil abusar de funciones secuenciales como el delay(); sin embargo, aquí quedó expuesta su grave deficiencia al congelar todo el equipo. Más allá del montaje del hardware, el gran aprendizaje radicó en cómo un correcto enfoque algorítmico y matemático sobre el reloj del procesador (guardando estados y diferenciales de tiempo con variables globales y millis()) permite crear interactividad de alta calidad. Además, asimilar que el software debe usarse como "amortiguador" de las imperfecciones del mundo físico (ruido del botón, fluctuaciones eléctricas) abre una perspectiva crítica y obligatoria para cualquier profesional abocado al diseño de sistemas embebidos e IoT.

10. Bibliografía / Referencias

- Freire Sánchez, L. A. (2023). Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico. ISBN 978-9942-636-22-5.
- Zailani, N. F. (2024). Getting started with Arduino and ESP32: A beginner's guide with Wokwi. Politeknik Mersing.
- Espressif Systems. (s. f.). ESP32 Series Datasheet. https://www.espressif.com/en/support/documents/technical-documents
- Espressif Systems. (s. f.). Arduino-ESP32 documentation. https://docs.espressif.com/projects/arduino-esp32/
- CodeMagic LTD. (2025). Wokwi – documentación. https://docs.wokwi.com
- Arduino. (2024). Arduino Language Reference. https://www.arduino.cc/reference/

11. Anexos

link git: https://github.com/IsmaelGonz/AP2-SistemasDigitales